FULL-STACK E-COMMERCE PLATFORM

Apple Boutique

A production e-commerce platform — Spring Boot API with a storefront wired to every endpoint.

Spring Boot 3.3 · Java 21 · PostgreSQL 16 · Flyway
Stateless JWT security · faceted search · stock reservations
Pluggable payments · coupons · reviews · analytics dashboard
Plain HTML/CSS/JS storefront — no framework, no build step

DOCUMENTATION — product & team onboarding
Version 2.0 · 10 October 2026
67 API operations · 48 paths · 84 automated tests · verified against PostgreSQL 16

Contents

Part I — Product & Architecture 3

Quick start 3

Architecture 4

Module map 5

Domain model 6

API surface 6

The storefront (front end) 9

Design decisions 10

Configuration 11

Testing 12

Deploying 12

Production checklist 12

Licence 13

Part II — Team Onboarding 14

1. First hour 14

2. The shape of the codebase 14

3. How a request flows — worked example 15

4. Conventions 16

5. Adding an endpoint 17

6. Adding a whole feature 18

7. Database changes 18

8. Gotchas we have already hit 18

9. Tests: what to write where 20

10. Where do I find…? 21

11. Before you open a pull request 21

12. Glossary 21

13. The storefront (front end) 22

PART I

Product & Architecture

A full-stack e-commerce platform: a storefront (plain HTML/CSS/JavaScript, served by the API itself) on top of a catalogue with faceted search, stock-aware cart, coupons, checkout with pluggable payments, fulfilment workflow, reviews, wishlist, an analytics dashboard and a merchant console.

Built with **Spring Boot 3.3**, **Java 21**, **PostgreSQL**, stateless **JWT** security and **Flyway** migrations. 67 documented REST operations across 48 paths, 84 automated tests, and a storefront wired to every one of them.

Quick start

```
docker compose up -d      # PostgreSQL 16 + MailHog (web inbox on :8025)
./mvnw spring-boot:run    # API on http://localhost:8080
```

Then open **http://localhost:8080/swagger-ui.html**, click **Authorize**, and paste a token from `POST /api/auth/login`.

No Docker? Point the app at any PostgreSQL instance:

```
export DB_URL=jdbc:postgresql://localhost:5432/ecommerce_db
export DB_USERNAME=postgres
export DB_PASSWORD=yourpassword
./mvnw spring-boot:run
```

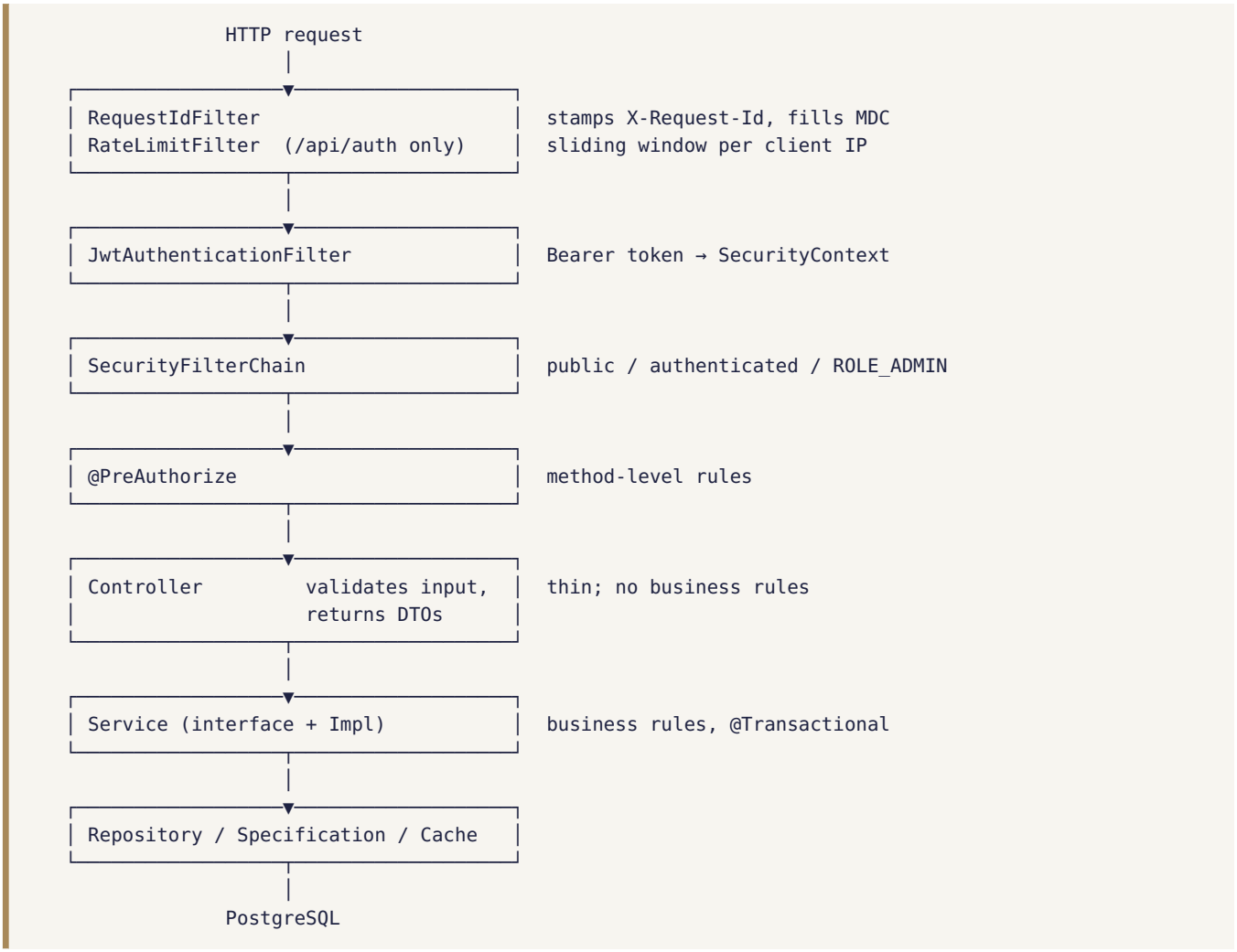
The schema is created by Flyway on first boot, then `DataInitializer` seeds roles, two accounts, sixteen products across eight boutique categories, and the `WELCOME10` and `APPLE10` coupons. Seeding is idempotent — restart as often as you like. Disable it with `--seed.enabled=false`.

Demo accounts

EMAIL	PASSWORD	ROLE
`admin@example.com`	`Admin@123`	`ROLE_ADMIN`
`customer@example.com`	`Customer@123`	`ROLE_USER`

> Change these (and the JWT secret) before anything reaches the internet. Disable seeding with `--seed.enabled=false`.

Architecture



Anything thrown anywhere becomes the standard `ApiError` JSON via `GlobalExceptionHandler`.

Principles the code follows

PRINCIPLE	WHAT IT LOOKS LIKE HERE
Package by feature	`catalog/product`, `order`, `payment` — a feature's code sits together
DTOs at the boundary	Entities never serialise to JSON; requests/responses are immutable records
One transaction per use case	`@Transactional` on services, never on controllers
Fail loudly and early	Typed exceptions carrying a machine-readable `ErrorCode`
Composition over branching	`PaymentGateway`, `FileStorageService`, `NotificationService` are interfaces
Explicit over magic	`Specification` for search instead of generated query methods

Module map

PACKAGE	RESPONSIBILITY
`common`	`ApiError`, `PageResponse`, `ErrorCode`, exceptions, `GlobalExceptionHandler`, `BaseEntity`
`config`	Security, caching, async, CORS, rate limiting, OpenAPI, typed `ApplicationProperties`
`security`	`JwtService`, `JwtAuthenticationFilter`, `CustomUserDetails`, entry point, access-denied handler
`user`	Accounts, roles, addresses, refresh tokens, registration and login
`catalog.category`	Category CRUD
`catalog.product`	Product CRUD, images, search, facets
`catalog.review`	Ratings, verified-purchase review, moderation
`cart`	Cart lines, live pricing, coupons on carts, reservation expiry job
`inventory`	Stock reservation, release, confirmation
`promotion`	Coupon validation and discount calculation
`order`	Checkout orchestration, order lifecycle, fulfilment
`payment`	`PaymentGateway` abstraction, mock provider, payment records, refunds
`wishlist`	Saved products
`analytics`	Admin dashboard aggregates
`storage`	File upload and serving
`notification`	Transactional email, event-driven and async
`seed`	Idempotent demo data

Storefront files (`src/main/resources/static`)

FILE	RESPONSIBILITY
`index.html`	Page skeleton; every section is a container the JS fills
`css/styles.css`	Design tokens (top of file) and all components
`js/config.js`	Brand copy that does not live in the database (nav, founder, hours, WhatsApp)
`js/api.js`	The API client: bearer tokens, single-flight refresh, one error shape
`js/ui.js`	Shared rendering helpers: INR formatting, escaping, toasts, placeholders
`js/store.js`	Tiny shared state: cart count, wishlist, categories
`js/catalog.js`	Search, facet filters, product grid, quick view, reviews
`js/cart.js`	Bag drawer, checkout, address entry, payment retry
`js/account.js`	Sign in/up, profile, addresses, orders, saved items

FILE	RESPONSIBILITY
`js/admin.js`	Merchant console: dashboard, orders, products, coupons, moderation
`js/main.js`	Bootstrap: static sections, hero, coverflow, appointment form
`assets/images/`	Product, category, hero and studio art (SVG placeholders; swap in photos)

There is no bundler, no framework, and no build step — the browser loads the files in order and the page is done.

Domain model

```
User ──< Address
 │
 ├── 1:1  Cart ──< CartItem >── Product ──< ProductImage
 │                                  │
 ├── *:* Role                       └──< Review
 │
 └──< Order ──< OrderItem
              └── 1:1 Payment

Category ──< Product
```

Stock: two counters, not one

`Product` carries `stock_quantity` (physical units) and `reserved_quantity` (units sitting in active carts).

```
available = stock_quantity - reserved_quantity
```

Adding to cart **reserves**, checkout **confirms** (both counters drop), removing from cart or cancelling an order **releases** (reserved drops, physical stays). A scheduled job (`ReservationExpiryJob`) releases reservations on baskets idle for longer than `app.inventory.reservation-ttl` (60 minutes by default), so abandoned carts cannot make stock unsellable forever.

Order lifecycle

```
PENDING ──► CONFIRMED ──► PROCESSING ──► SHIPPED ──► DELIVERED
   │            │             │
   └────────────┴─────────────┴──────────► CANCELLED   (stock returned)
                                  └──►  REFUNDED
```

The permitted moves live in exactly one place, `OrderStatus.canTransitionTo`, and are unit-tested. Payments drive `PENDING → CONFIRMED`; cash on delivery confirms without capturing money.

API surface

All paths are prefixed with `/api`. public, authenticated, admin.

Authentication

METHOD	PATH	NOTES
POST	`/auth/register`	Returns access + refresh token pair
POST	`/auth/login`	Same
POST	`/auth/refresh`	Rotates; the presented refresh token dies immediately
POST	`/auth/logout`	Revokes the refresh token

Access tokens live 15 minutes, refresh tokens 7 days. Refresh tokens are stored **hashed** and rotated on every use, so a stolen one stops working as soon as the real client uses it again.

Catalogue

METHOD	PATH	AUTH	NOTES
GET	`/catalog/products`		`keyword`, `categoryId`, `brand`, `minPrice`, `maxPrice`, `minRating`, `inStockOnly`, `featuredOnly`, `page`, `size`, `sort`
GET	`/catalog/products/facets`		Category, brand, price-bucket and rating counts
GET	`/catalog/products/{id}`		
GET	`/catalog/products/slug/{slug}`		
GET	`/catalog/products/{id}/reviews`		Approved reviews
POST	`/catalog/products/{id}/reviews`		One per customer per product
PUT/DELETE	`/catalog/products/{id}/reviews/{reviewId}`		Own reviews only
GET	`/catalog/categories`		
GET	`/catalog/categories/{id}` `/slug/{slug}`		
POST/PUT/DELETE	`/catalog/products` `/catalog/categories`		
POST	`/catalog/products/{id}/images`		`multipart/form-data`, field `file`
DELETE	`/catalog/products/{id}/images/{imageId}`		
GET	`/catalog/products/low-stock`		`?threshold=5`

Shopping

METHOD	PATH	NOTES
GET	`/cart`	Lines plus live `subtotal / discount / shipping / total`
POST	`/cart/items`	`{ productId, quantity }` — reserves stock
PUT	`/cart/items/{productId}?quantity=3`	Releases or reserves the delta
DELETE	`/cart/items/{productId}`	Releases the reservation
DELETE	`/cart`	Empties the cart
POST	`/cart/coupon`	`{ "code": "WELCOME10" }`
DELETE	`/cart/coupon`	
GET/POST/DELETE	`/addresses` `/addresses/{id}`	
PATCH	`/addresses/{id}/default`	
GET/POST/DELETE	`/wishlist/{productId}`	
POST	`/wishlist/{productId}/move-to-cart`	
GET/PUT/PATCH	`/users/me` `/users/me/password`	Profile

Orders and payments

METHOD	PATH	NOTES
POST	`/orders/checkout`	`{ addressId, paymentMethod, notes, idempotencyKey }`
GET	`/orders` `/orders/{id}` `/orders/number/{n}`	Own orders only
PATCH	`/orders/{id}/cancel`	While `PENDING` or `CONFIRMED`
GET	`/orders/{id}/payment`	
POST	`/orders/{id}/payment/retry`	After a decline

`paymentMethod` is one of `CARD`, `UPI`, `NETBANKING`, `WALLET`, `COD`.

Admin

METHOD	PATH
GET/DELETE	`/admin/users` `/admin/users/{id}`
GET	`/admin/orders`
PATCH	`/admin/orders/{id}/status`

METHOD	PATH
POST	`/admin/orders/{id}/refund`
GET/POST/PUT/DELETE	`/admin/coupons` `/admin/coupons/{id}`
GET	`/admin/analytics/dashboard`
GET	`/catalog/products/{id}/reviews/moderation/pending`
PATCH	`/catalog/products/{id}/reviews/moderation/{reviewId}?status=APPROVED`

Operations

`GET /actuator/health`, `/actuator/info`, `/actuator/metrics`, `GET /api/files/{filename}` (uploaded images), Swagger UI and `/v3/api-docs`.

The storefront (front end)

A working front end is included and **already wired to every endpoint** — no build step. It is plain HTML, CSS and JavaScript served by the API from `src/main/resources/static`, so the page and the API share one origin (no CORS to configure, works offline, no deploy step for the front end).

Run it: start the app (Quick start above), then open **http://localhost:8080/**.

What it does — this is the real API, not a mock: - Browse: live catalogue with faceted filters (category, designer, rating, price), keyword search, sort, pagination. Products, stock and ratings come from `GET /api/catalog/products`. - Quick view: product detail, image, live reviews, and write-a-review. - Bag & checkout: stock is reserved server-side the moment you add an item; apply a real coupon, add an address, choose Card or COD, and the mock gateway captures the payment. The bag, coupon, address and order are all in the database. - Account: sign in / sign up (real JWT with rotating refresh token), manage addresses, view orders, cancel, retry a declined payment, and move wishlist items into the bag. - Merchant console (admin only): revenue dashboard, the order lifecycle (process → ship → deliver, with legal-transition validation), product + category + coupon CRUD, image upload, and review moderation.

Try it as a shopper: 1. Add a few pieces to the bag, apply coupon `APPLE10` at checkout, and place an order with Card. 2. Sign in as `customer@example.com` / `Customer@123` to see the order under **My Orders**.

Try it as the merchant: 1. Sign in as `admin@example.com` / `Admin@123`. 2. The **Admin** nav link reveals the console: watch revenue appear on the dashboard, advance the order through Processing → Shipped, add a product or coupon, and moderate a review.

Swap in real photography: the seed references `assets/images/p-01.svg` … `p-16.svg`, category tiles `cat-*.svg`, and the hero/founder/studio/login art. Drop real files with the same names (any of the formats the upload accepts — JPEG/PNG/WebP/GIF) into `src/main/resources/static/assets/images/` and they replace the generated placeholders. The seed reads the image filename, so no code change is needed.

API contract (for any other client)

The storefront's client lives in `static/js/api.js` and is a useful reference. If you build a separate front end:

Base URL: `http://localhost:8080` (dev). Point the UI at it via an env var, not a hard-coded string.

Auth flow. `POST /api/auth/login` → store `accessToken` (memory or `sessionStorage`) and `refreshToken`. Send `Authorization: Bearer <accessToken>` on every call. On a `401`, call `/api/auth/refresh` **once** and retry the original request — the refresh token is rotated, so a second concurrent refresh would fail. On a bad refresh, drop both tokens and send the user to login.

CORS. Allowed origins are configured by `app.security.cors.allowed-origins` (defaults to `localhost:3000`, `:5173`, `:4200`). Add your deployed origin there. Credentials are allowed, and `Authorization` is exposed.

Paging. List endpoints return:

```
{ "content": [], "page": 0, "size": 24, "totalElements": 8, "totalPages": 1, "first": true, "last":
true }
```

Sort with `?sort=price,asc` (or `?sort=name,desc`).

Errors. Always this shape — branch on `code`, never on `message`:

```
{
  "timestamp": "2026-10-08T10:30:00Z",
  "traceId": "2Yci1dR6cUdE3ZJi",
  "status": 400,
  "code": "VALIDATION_FAILED",
  "message": "The request payload is invalid",
  "path": "/api/cart/items",
  "fieldErrors": { "quantity": "must be at least 1" }
}
```

STATUS	MEANING
401	No token, bad token, or expired and no refresh available → log in
403	Logged in but not permitted → hide the control
404	`*_NOT_FOUND` → show "not found"
409	`INSUFFICIENT_STOCK`, `*_ALREADY_EXISTS`, `ORDER_NOT_CANCELLABLE` → show the message inline
402	`PAYMENT_FAILED` → offer another payment method

Money is a decimal number with two places; treat it as a string on the client if your language loses precision. Dates are ISO-8601 UTC.

Suggested page load sequence: `GET /catalog/categories` + `GET /catalog/products?...` + `GET /catalog/products/facets?...` in parallel, then `GET /cart` once a token exists.

Design decisions

DECISION	WHY
Stateless JWT, no sessions	Scales horizontally; safe for browsers and mobile alike
CSRF disabled	CSRF defends cookie sessions; there is no ambient credential here
Refresh tokens hashed + rotated	A stolen refresh token dies on first legitimate reuse

DECISION	WHY
Stock reserved at add-to-cart, not checkout	Two shoppers cannot both buy the last unit
`OrderItem` snapshots name, SKU, price and address	Order history survives later catalogue and address edits
Soft delete for products (`active = false`)	Past orders keep referencing a real product row
`BigDecimal` for money	Binary floating point cannot represent `0.10`
Cached catalogue reads with explicit eviction	Catalogue is read far more than written
Denormalised `average_rating` on the product row	Lets listings filter and sort by rating without a join per row
Flyway migrations + `ddl-auto: validate`	Schema changes are reviewable; Hibernate cannot drift from them
Ownership inside the query (`findByIdAndUserId`)	One less way to leak another customer's data
Request correlation id (`X-Request-Id`)	Support can go from a user report straight to the log line

Configuration

Everything lives under `app.*` in `application.yml` and is overridable by environment variable.

PROPERTY	ENV VAR	DEFAULT
`spring.datasource.url`	`DB_URL`	`jdbc:postgresql://localhost:5432/ecommerce_db`
`spring.datasource.username`	`DB_USERNAME`	`postgres`
`spring.datasource.password`	`DB_PASSWORD`	`postgres`
`app.security.jwt.secret`	`JWT_SECRET`	dev key — **replace it**
`app.security.jwt.access-token-ttl`	`JWT_ACCESS_TTL`	`15m`
`app.security.jwt.refresh-token-ttl`	`JWT_REFRESH_TTL`	`7d`
`app.security.cors.allowed-origins`	`CORS_ORIGINS`	localhost dev origins
`app.security.rate-limit.requests-per-minute`	—	`60` (on `/api/auth/*` only)
`app.storage.upload-dir`	`UPLOAD_DIR`	`./uploads`
`app.storage.base-url`	`STORAGE_BASE_URL`	`http://localhost:8080/api/files`
`app.inventory.reservation-ttl`	`RESERVATION_TTL`	`60m`
`app.notifications.enabled`	—	`true`
`seed.enabled`	—	`true`

Generate a real JWT secret:

```
openssl rand -base64 48
```

Email. Set `spring.mail.host` (MailHog in `docker-compose.yml` listens on `1025` with a web inbox on `8025`). With no mail server configured, notifications are logged instead of sent — a broken SMTP server can never fail an order.

Testing

```
./mvnw test
```

84 tests. No external services required.

GROUP	WHAT IT COVERS
Unit (Mockito)	Pricing rules, coupon maths, cart reservations, inventory, JWT, slugs, order transitions
Integration (`@SpringBootTest` + MockMvc + H2)	Auth flows incl. refresh rotation, the whole browse→cart→coupon→checkout→fulfil→review journey, oversell prevention, cash on delivery, authorisation boundaries

Integration tests run on in-memory H2 with `ddl-auto: create-drop`, so the test schema always matches the entities.

Deploying

```
./mvnw clean package -DskipTests
docker build -t ecommerce-api:2.0 .

docker run -p 8080:8080 \
  -e DB_URL=jdbc:postgresql://db:5432/ecommerce_db \
  -e DB_USERNAME=postgres \
  -e DB_PASSWORD=secret \
  -e JWT_SECRET="$(openssl rand -base64 48)" \
  -e CORS_ORIGINS=https://shop.example.com \
  -e SEED_ENABLED=false \
  -v ecommerce-uploads:/app/uploads \
  ecommerce-api:2.0
```

The `Dockerfile` is multi-stage: build with a full Maven + JDK image, ship only the jar on a JRE Alpine image running as a non-root user.

Production checklist

- [ ] Replace `app.security.jwt.secret` with `openssl rand -base64 48`, injected as a secret
- [ ] Change the seeded admin and customer passwords, or boot with `--seed.enabled=false`
- [ ] Restrict `app.security.cors.allowed-origins` to real front-end origins
- [ ] Terminate TLS so tokens are never sent in clear text

- [ ] Move rate limiting to the API gateway or a Redis-backed limiter (the in-process one does not span instances)
- [ ] Replace `LocalFileStorageService` with S3/GCS for multi-instance deployments
- [ ] Replace the in-process Caffeine cache with Redis if you run more than one instance
- [ ] Add `spring-boot-starter-actuator` auth on `/actuator/*` (currently only health and info are exposed)
- [ ] Configure real SMTP credentials for order emails
- [ ] Back up PostgreSQL; Flyway migrations are forward-only
- [ ] Implement `PaymentGateway` for your provider and annotate it `@Primary`

Licence

MIT.

New to the codebase? Read **ONBOARDING.md** next — it walks through the layout, the conventions, and how to add a feature.

PART II

Team Onboarding

Welcome. This document exists so that your first change lands the same afternoon you clone the repo. It explains why the code is shaped the way it is, not just what each file does.

Read **Part I — Product & Architecture** first for what the product does and how to run it. Come back here when you start changing things.

---

1. First hour

```
git clone <repo> && cd ecommerce-api
docker compose up -d
./mvnw spring-boot:run
```

Then:

1. Open **http://localhost:8080/** — the storefront. Browse, add something to the bag, check out with coupon `APPLE10`. This is the fastest way to understand what the product is.
2. Open http://localhost:8080/swagger-ui.html — the fastest way to understand the surface area.
3. Log in as `admin@example.com` / `Admin@123` via `POST /api/auth/login`, click **Authorize**, paste the token.
4. Call `GET /api/catalog/products`. You should see the sixteen seeded boutique products.
5. Sign in as the admin in the storefront: the **Admin** nav link reveals the merchant console.
6. Run `./mvnw test`. 84 tests, all green, no database needed.

IDE setup

IntelliJ IDEA is what most of us use. Two things are mandatory:

- **Enable annotation processing** (Settings → Build → Compiler → Annotation Processors). Lombok generates getters and builders at compile time; without this the IDE shows hundreds of false errors.
- **Install the Lombok plugin.**

Use the bundled `./mvnw` rather than a system Maven — it pins 3.9.9 so everyone builds identically.

---

2. The shape of the codebase

Code is organised **by feature, not by layer**. Everything about products lives under `catalog/product`; everything about orders lives under `order`.

```
com.example.ecommerce
├── common/      shared plumbing: errors, paging, base entity
├── config/      Spring configuration and typed settings
├── security/    JWT, filters, principal
├── user/        accounts, roles, addresses, login
├── catalog/
│   ├── category/
│   ├── product/
│   └── review/
├── cart/        cart lines, pricing, reservations
├── inventory/   stock counters
├── promotion/   coupons
├── order/       checkout and fulfilment
├── payment/     gateway abstraction + mock
├── wishlist/
├── analytics/
├── storage/     file upload
├── notification/
└── seed/        demo data
```

A feature folder typically contains:

```
product/
├── Product.java               entity
├── ProductImage.java
├── ProductRepository.java     database access
├── ProductSpecification.java  dynamic query building
├── ProductService.java        interface
├── ProductServiceImpl.java    business logic
├── ProductController.java     HTTP
└── dto/                       records: requests and responses
```

Why by feature? When you are asked to change how discounts work, you open one folder. When a team grows, features become ownership boundaries.

If you are used to `controllers/`, `services/`, `repositories/` at the top level, this takes a day to get used to and then you will not want to go back.

Everything in this section and beyond is about the **API**. The **storefront** (the HTML/CSS/JavaScript front end) lives outside the Java packages, in `src/main/resources/static`, and is covered in §13.

---

3. How a request flows — worked example

Let's trace `POST /api/cart/items` with `{ "productId": 7, "quantity": 2 }`.

1. `RequestIdFilter` (config/WebConfig) — reads or generates `X-Request-Id`, puts it in the MDC so every log line for this request carries it, and sets it on the response.

2. `RateLimitFilter` — only registered for `/api/auth/*`, so it is skipped here.

3. `JwtAuthenticationFilter` — sees `Authorization: Bearer …`, verifies the signature and expiry, loads the user, and puts a `CustomUserDetails` into the `SecurityContext`. If anything is wrong it clears the context and continues; the security rules decide the outcome, not the filter.

4. `SecurityFilterChain` — `/api/cart/**` is not in the public list and is not `/api/admin/**`, so it falls to `anyRequest().authenticated()`. No authentication → `AuthEntryPoint` writes a 401.

5. `CartController.addItem`

```
@PostMapping("/items")
public ResponseEntity<CartResponse> addItem(@Valid @RequestBody AddItemRequest request) {
    return ResponseEntity.ok(cartService.addItem(SecurityUtils.currentUserId(), request));
}
```

That is the whole controller method. It validates the body, resolves who is calling from the security context, and delegates. Controllers never contain business rules — that is what makes services testable without HTTP.

6. `CartServiceImpl.addItem` — the interesting part:

```
Cart cart = cartFor(userId);
Product product = requirePurchasable(request.productId());

cart.findItem(product.getId()).ifPresentOrElse(
    existing -> {                                  // already in the basket
        inventoryService.reserve(product.getId(), request.quantity());
        existing.setQuantity(existing.getQuantity() + request.quantity());
        existing.setUnitPrice(product.getPrice());
    },
    () -> {                                        // new line
        inventoryService.reserve(product.getId(), request.quantity());
        cart.addItem(CartItem.builder()
                .product(product)
                .quantity(request.quantity())
                .unitPrice(product.getPrice())
                .build());
    });

return toResponse(cartRepository.save(cart));
```

Note what happens: stock is **reserved** the moment an item is added, and the line records the price at that moment. Both matter — see §8.

7. `CartResponse` — a record with the lines and freshly computed totals. The controller returns it; Jackson serialises it.

8. Something went wrong? Any exception is caught by `GlobalExceptionHandler` and rendered as the standard `ApiError`. You never write try/catch in a controller.

---

4. Conventions

These are not style preferences; they keep the codebase predictable.

DTOs are records

```
public record AddItemRequest(
        @NotNull Long productId,
        @NotNull @Min(1) Integer quantity
) {}
```

Immutable, no builder needed, validation reads cleanly. Jackson handles them natively.

Do **not** return entities from controllers. The moment you do, adding a column to a table can silently expose it — including things like `passwordHash`.

Responses are projections, not entities

Same rule, other direction. `UserResponse.from(user)` picks the fields deliberately.

Services: interface + Impl

Yes, it is one extra file. It buys you a stable seam for mocking, and it keeps implementation details (caching annotations, transaction boundaries) off the contract.

Errors carry a code, not a sentence

```
throw new ResourceNotFoundException(ErrorCode.PRODUCT_NOT_FOUND);
throw new ConflictException(ErrorCode.INSUFFICIENT_STOCK,
        "Only %d unit(s) of '%s' are available".formatted(available, name));
```

Clients branch on `code`. Messages are for humans and are free to change. Add a new `ErrorCode` rather than reusing a vague one.

Transactions on services

```
@Transactional          // writes
@Transactional(readOnly = true)   // reads — lets Hibernate skip dirty checking
```

One service call = one transaction. Never put `@Transactional` on a controller.

Settings are typed

Add a field to `ApplicationProperties`, not a new `@Value` annotation. All configuration is then visible in one place and validated at startup.

Javadoc on public APIs

Public methods get a short Javadoc saying what they do and what they return. Comments inside methods explain **why**, never **what** — the code already says what.

```
// Good: explains a non-obvious constraint
// Images are not fetched here: Hibernate cannot fetch two List collections
// in one query (MultipleBagFetchException). @BatchSize covers it instead.

// Bad: restates the code
// Loop over the items
for (CartItem item : cart.getItems()) {
```

---

5. Adding an endpoint

Say you need `GET /api/catalog/products/{id}/similar`.

1. **Response DTO** — `catalog/product/dto/ProductSummary.java` probably already fits. If not, add a record.
2. **Service method** — declare on `ProductService`, implement in `ProductServiceImpl`. Repository query goes in `ProductRepository`.
3. **Controller method** — in `ProductController`, with `@Operation` for the docs.

4. **Security** — is it public, authenticated, or admin? Public: add to `PUBLIC_PATHS` in `SecurityConfig` if the prefix is not already there. Admin: `@PreAuthorize("hasRole('ADMIN')")`.
5. **Test** — one integration test that calls it over MockMvc and asserts status plus a couple of fields.

That is it. No XML, no wiring, no bean registration.

---

6. Adding a whole feature

Follow the folder template in §2. Then:

- **New table?** Add a Flyway migration — see §7.
- **Needs to react to an existing event?** Publish a record (like `OrderPlacedEvent`) and listen with `@TransactionalEventListener(phase = AFTER_COMMIT)` so your work runs only after the original transaction commits.
- **Calls an external system?** Define an interface (like `PaymentGateway`), ship a working implementation, and let tests mock the interface.
- **Slow or remote?** Do it on the async executor: `@Async(AsyncConfigTaskExecutor.NAME)`.

---

7. Database changes

Schema is owned by **Flyway**, in `src/main/resources/db/migration/`.

```
V1__init_schema.sql
V2__add_product_video_url.sql     ← next one
```

Rules:

- **Never edit an applied migration.** Add a new one. Flyway checksums them.
- `ddl-auto` is `validate`: Hibernate checks that entities match the schema and **refuses to start** if they drift. That is deliberate — it catches "I added a field and forgot the migration" at boot instead of at 2am.
- Money is `NUMERIC(12,2)`. Timestamps are `TIMESTAMPTZ`. Booleans are `BOOLEAN`.
- Watch out for SQL keywords as column names (`value`, `type`, `order`, `user`) — we hit this with `coupons.value` and renamed it to `discount_value`.

Local dev: Flyway runs automatically. Tests: Flyway is off and Hibernate builds the schema from the entities, so `./mvnw test` never needs a database.

---

8. Gotchas we have already hit

Read this section before your first review — every one of these cost someone an afternoon.

Hibernate cannot fetch two `List` collections in one query. This throws `MultipleBagFetchException`. Our cart query used to fetch both `Cart.items` and `Product.images` and blew up at runtime. Fix: fetch one, and let `@BatchSize` on the other association turn N+1 into 1 query per 20 rows. If you add a `join fetch`, run the integration tests.

A 403 can come back as a 401. Spring Security's default access-denied handler calls `sendError(403)`, which forwards to `/error`; that dispatch has no security context, so it emerged as a 401 with `path: "/error"`. We fixed it with `ApiAccessDeniedHandler`, which writes the JSON directly. If you touch `SecurityConfig`, re-check that a logged-in non-admin gets **403** from an admin endpoint.

`@ConditionalOnMissingBean` **on a** `@Component` **does not work.** It is evaluated during component scanning and depends on order. The mock payment gateway used it and never registered, so the app failed to start. Ship the default unconditionally; a real provider marks itself `@Primary`.

`Specification.and(null)` **throws.** `ProductSpecification.combine` handles the nulls — use it rather than chaining `and()` yourself.

Cached availability goes stale unless the stock move announces itself. The product cache holds `availableQuantity` with a 10-minute TTL. Nothing used to evict it when a cart reserved units, so shoppers saw stock from when the entry was cached and the last unit could appear free to two baskets at once. The fix: `InventoryServiceImpl` publishes `ProductStockChangedEvent` after every mutation and `ProductServiceImpl` drops that product's cache entries on `AFTER_COMMIT`. **If you move stock numbers anywhere else, publish the event** — that is why `returnOrderStock` now calls `inventoryService.restock` instead of setting the field directly.

Lazy loading outside a transaction throws `LazyInitializationException`**.** `open-in-view` is `false` on purpose. If you need a collection in a response, fetch it in the query (`join fetch`) or map it inside the `@Transactional` service method.

Reservation arithmetic is easy to get wrong. Adding to a cart that already has the item reserves only the delta. Lowering a quantity releases only the difference. The unit tests in `CartServiceImplTest` pin this down — keep them green.

Do not build a LIKE pattern with `concat` **in JPQL.** `concat('%', lower(:keyword), '%')` compiles, and works on H2, but on PostgreSQL Hibernate binds `:keyword` as BINARY inside the `concat` and the driver rejects it: `function lower(bytea) does not exist`. Build the pattern in Java (`ProductServiceImpl.likePattern`) and pass a non-null value — `"%"` means "match everything". Prefer a `Specification` for anything else that filters on optional criteria.

H2 cannot validate the Flyway schema. The tests run on H2 in PostgreSQL mode, but H2 does not enforce PostgreSQL's type rules: `reviews.rating` was `SMALLINT` in the migration while the entity field was `int`, and only `ddl-auto: validate` against a real PostgreSQL 16 caught it. **Before merging a migration, boot the app against a real PostgreSQL.** `docker compose up -d postgres` is enough.

Unknown paths used to return 500. A request to a route with no handler falls through to Spring's static-resource handler and surfaces as `NoResourceFoundException`, which the catch-all reported as `INTERNAL_ERROR` — so a mistyped URL looked like an outage. `GlobalExceptionHandler` now maps it to `404 RESOURCE_NOT_FOUND`. Note that an anonymous request to an unknown `/api/**` path still gets **401**, which is deliberate: we do not leak the endpoint map to strangers.

The mock payment gateway has to be able to say no. `MockPaymentGateway` originally hardcoded `alwaysSucceed = true`, so the declined-payment path — and therefore payment retry — was unreachable code that nobody had ever run. It is now driven by `app.payments.mock-always-succeed` (`MOCK_PAYMENT_ALWAYS_SUCCEED`, default `true`). Remember that a declining gateway is the normal way into the retry flow, so exercise that path before shipping anything that touches payments.

Counts and delete guards must respect soft deletes. `ProductServiceImpl.retire` is a soft delete: it sets `active = false` and leaves the row. Any count that gates an operation or is shown to a user there-

fore has to filter on `active`, which is why `ProductRepository` exposes `countByCategoryIdAndActiveTrue` and not plain `countByCategoryId`. A category containing only retired products used to be undeletable forever. **The same rule applies to anything else you count** — ask "should archived rows count?" before writing `countBy…`.

A payment retry must update the existing row, never insert a second one. `payments.order_id` is unique: one order has exactly one payment row, across every attempt. `PaymentServiceImpl.charge()` used to build a fresh `Payment` unconditionally, so retrying a declined payment issued an INSERT against that constraint and the endpoint returned **500 every single time** — the flow had never worked. `charge()` now looks the row up with `findByOrderId` and reuses it, clearing `gatewayReference`, `failureReason` and `capturedAt` so the old failure cannot masquerade as the current state. `PaymentRetryTest` pins this: it asserts the row id is unchanged and that only one payment exists for the order.

---

9. Tests: what to write where

SITUATION	TEST TYPE	EXAMPLE
Pure logic, no collaborators	Unit	`PricingPolicyTest`, `OrderStatusTest`
Business rules with collaborators	Unit + Mockito	`CartServiceImplTest`, `CouponServiceImplTest`
"Does the whole thing actually work"	Integration	`ShoppingFlowTest`
Security rules	Integration	`AuthorizationTest`
A path the real dependencies cannot reach	Integration + stub `@Primary` bean	`PaymentRetryTest`

Rule of thumb: every new business rule gets a unit test. Every new endpoint gets at least one integration test that goes through MockMvc. Integration tests are slower (they boot Spring) but they are the ones that catch wiring mistakes — every bug in §8 was found by running the app, not by reading it: four surfaced from integration tests, and three more only appeared once the app was pointed at a real PostgreSQL.

When a real dependency cannot produce the case you need, replace it. The mock gateway either always succeeds or (with `mock-always-succeed=false`) always declines, so neither setting can walk decline, then recover. `PaymentRetryTest` supplies its own `@Primary PaymentGateway` from a `@TestConfiguration` that declines only when the idempotency key contains `fail`. Two lessons: keep stub state **out** of shared bean state (a counter that leaks between tests makes them order-dependent — drive the behaviour from an argument instead), and re-read entities from the repository after a service call, because the service works on its own managed instance and your local reference is stale.

Run a single test:

```
./mvnw test -Dtest=CartServiceImplTest
./mvnw test -Dtest='ShoppingFlowTest#fullJourney'
```

---

10. Where do I find...?

I WANT TO…	GO TO
Add or change an endpoint	`*/Controller.java`
Change a business rule	`*/ServiceImpl.java`
Add a validation rule	the request record in `*/dto/`
Add an error code	`common/exception/ErrorCode.java`
Change who can call what	`config/SecurityConfig.java` + `@PreAuthorize`
Add a config setting	`config/ApplicationProperties.java` + `application.yml`
Change the schema	`src/main/resources/db/migration/`
Change shipping or discount maths	`cart/PricingPolicy.java`, `promotion/CouponServiceImpl.java`
Change how stock is held	`inventory/InventoryServiceImpl.java`, `cart/ReservationExpiryJob.java`
Plug in a real payment provider	implement `payment/PaymentGateway.java`, annotate `@Primary`
Change demo data	`seed/DataInitializer.java`
Change email content	`notification/NotificationServiceImpl.java`
Add a scheduled job	`@Scheduled` component (see `ReservationExpiryJob`), enabled by `@EnableScheduling`

11. Before you open a pull request

- [ ] `./mvnw test` is green
- [ ] New public methods have Javadoc
- [ ] New errors use a specific `ErrorCode`, not `INTERNAL_ERROR`
- [ ] Any new query was checked for N+1 (run with `logging.level.org.hibernate.SQL=DEBUG` and count the queries)
- [ ] Schema change has a migration, and `ddl-auto: validate` still passes
- [ ] New endpoint appears correctly in Swagger with a description
- [ ] Authorisation was tested for all three cases: anonymous, wrong role, right role
- [ ] No entity is returned directly from a controller
- [ ] No commented-out code, no `System.out.println`, no `TODO` without a ticket reference

12. Glossary

TERM	MEANING HERE
Reservation	Units held by an active cart, not yet sold
Available	`stock_quantity - reserved_quantity`

TERM	MEANING HERE
COD	Cash on delivery — the order is confirmed without capturing money
Verified purchase	Review written by someone who actually bought the product
Facet	A filter option plus how many results it would give
Slug	URL-safe version of a name, e.g. `wireless-headphones`
Soft delete	Setting `active = false` rather than deleting the row
Rotation	Issuing a new refresh token and invalidating the one just used

13. The storefront (front end)

The front end is plain HTML, CSS and JavaScript in `src/main/resources/static`. There is no framework, no bundler, and no build step — the browser loads the files and the page is done. Because it is served by the API itself, the page and the API share one origin, so there is no CORS to configure and nothing extra to deploy.

How it is wired

The page (`index.html`) is mostly empty containers. On `DOMContentLoaded`, `main.js` boots nine modules, each of which fills its section and talks to the API through a single client:

```
index.html ── static containers
   │
   └─ main.js (bootstrap)
        ├── config.js   brand copy not in the DB (nav, founder, hours, WhatsApp)
        ├── api.js      THE client: tokens, refresh, one error shape
        ├── ui.js       helpers: ₹ formatting, HTML escaping, toasts, placeholders
        ├── store.js    tiny shared state: cart count, wishlist, categories
        ├── catalog.js  search, facets, product grid, quick view, reviews
        ├── cart.js     bag drawer, checkout, address, payment retry
        ├── account.js  sign in/up, profile, addresses, orders, saved items
        └── admin.js    merchant console: dashboard, orders, products, coupons
```

Everything that changes when the data changes goes through `api.js`. There are no other `fetch` calls, no `XMLHttpRequest`, and no hard-coded API URLs. If you find yourself adding a `fetch(` outside `api.js`, stop and add a method to the client instead — it is the one place that knows how to attach the token, refresh it, and turn an `ApiError` into something the UI can read.

The two things worth understanding

Token refresh is single-flight. The backend rotates refresh tokens — presenting one spends it and returns a fresh pair. If two requests expired at the same time and each tried to refresh, the loser's token would already be revoked and the user would be logged out for no reason. So `api.js` funnels every 401 through one shared in-flight promise: the first to arrive refreshes, the others await the same result, and then everyone retries the original call with the new access token. Keep it that way if you touch `api.js`.

All interpolated values are escaped. The grid and the quick view build HTML from server data, so every value passes through `UI.esc`. Never interpolate a product name, a review comment, or an author name into `innerHTML` raw.

How to develop it

1. `docker compose up -d && ./mvnw spring-boot:run`
2. Open http://localhost:8080/ and open your browser's DevTools → Network tab. Every API call the page makes is right there, which is the fastest way to learn which endpoint a feature uses.
3. Edit the file in `static/`, refresh the browser. The API copies `static/` into `target/classes` on `compile`, so a **running** Spring Boot dev server picks up static changes without a restart; if it does not, restart the app.
4. You do not need a database reset to try most things — the seeded catalogue, the two demo accounts and the two coupons are enough. To reset to the seed, drop the `ecommerce` database and restart.

There is no JS test runner in the repo. The honest ways to verify a change are to exercise the flow in the browser and to run `./mvnw test`, which still guards the API contract the front end depends on. If you want a regression net for a specific flow, the cleanest place is a `ShoppingFlowTest`-style integration test on the endpoints it calls.

Swapping in real photography

The seed references image **filenames**, not URLs, so swapping art is a file operation, not a code change. Drop real files (JPEG/PNG/WebP/GIF) with the same names into `static/assets/images/`:

NAME	USED FOR
`p-01.svg` … `p-16.svg`	The sixteen seeded products
`cat-*.svg`	Category tiles (`cat-lehenga`, `cat-saree`, …)
`hero-1.svg`, `hero-2.svg`	Hero slider
`look-1.svg` … `look-7.svg`	Trending coverflow
`founder.svg`, `promise.svg`, `studio.svg`, `login.svg`, `logo.svg`	Brand sections

Any image the backend uploads through the admin console is stored under `/api/files/...` and served back through that route — the product's `primaryImageUrl` is whatever the first stored image is, so an uploaded photo replaces the seed art automatically.

Things that are intentionally not in the API

A few storefront features have no backend by design, and they are wired to the only sensible stand-in:

- **Newsletter** — no subscribers table. The form tells the user to use `APPLE10` at checkout, which is a real, server-validated coupon.
- **Appointment booking** — the form opens WhatsApp with the details pre-filled. That is the actual booking channel for the studio.

If a ticket ever says "add real email capture", that is a backend change (a `subscription` table + endpoint) plus a two-line change in `main.js` — but until then, do not build a client-side-only fake that pretends it persisted.

Questions? Ask in `#ecommerce-api`. If the answer turns out to be "we don't have a convention for that", add it to this file in the same pull request.